

Agent 开发周报 260408

刘德辰

2026 年 4 月 8 日

1 结构化报告数据源继续统一到 MCP：从车辆扩到驾驶员/线路，并统一最新分区策略

背景：上周车辆报告已经开始切到 MCP 画像数据源，但当时主要集中在车辆这一类报告，驾驶员、线路和报告 prompt 里的数据源说明还没有完全统一。本周这条线继续推进，目标是不再让不同报告类型各自维护一套取数方式、分区策略和缺数处理逻辑。

方案说明：本周把结构化报告数据源配置继续抽象成统一入口，分别把驾驶员报告、线路报告切到 MCP 画像工具，同时让车辆画像查询也经由统一 provider/gateway 出口处理。这样一来，报告 worker、structured lookup 和数据源说明就能围绕同一套配置收敛，而不是散落在多处临时判断。

功能说明：

- 将驾驶员报告切到 MCP 画像数据源，补上 driver profile 的统一读取封装
- 将线路报告切到 MCP 画像数据源，补上 route profile 的统一读取封装
- 将车辆画像查询改为通过 provider/gateway 统一访问，减少 Runtime 内直接区分 provider 的分支
- 统一 structured report prompt data sources 与数据源配置，减少报告技能、worker 和路由各自维护来源说明的情况
- 主画像类报告工具默认查询最新 partition，同时保留显式指定日期分区的能力
- 对齐车辆画像 ppartition 参数、lookup trace metadata 和 worker prompt UTF-8 编码处理，降低联调排查成本

2 Mock MCP 联调链路从“能模拟”推进到“可替代调试”：切换下沉、样本补齐与返回结果对齐

背景：随着报告链路越来越依赖 MCP，单纯“有一个 mock”已经不够用了。如果 mock 的切换入口、返回结构和样本文档彼此不一致，本地调试和反复验证时仍然会遇到大量伪问题，排查成本会非常高。

方案说明：本周把 mock MCP 的切换逻辑从零散调用点集中到 gateway layer，并默认采用 mock provider mode，先把“开发环境可稳定联调”这件事做好。然后继续补齐 mock registry、fixtures、generated samples 和文档样例，让 probe/list/call/debug 都能走一套完整协议结构。

功能说明：

3 ROUTER 统一判定继续推进：去掉场景提示和硬快捷分支，让澄清、追问和查询都回到统一判定2

- 将 mock MCP 开关下沉到 gateway layer，worker、research 和通用 MCP 调用统一复用同一套入口
- 默认 provider mode 调整为 mock，便于本地开发、反复验证和离线联调
- 新增并整理 mock MCP registry、matchers、fixtures 与 generated doc fixtures，覆盖车辆/驾驶员/线路主画像及列表查询
- 让 mock probe、tools/list、tools/call、detail/debug 都能返回完整协议结构，方便直接替代真实 MCP 做问题定位
- 对齐 mock 返回与 usable samples 文档，减少“文档里能查、mock 里没有”或字段结构不一致的问题
- 清理 Agent API 文档 BOM 等编码噪声，避免样本和接口文档再出现乱码干扰

3 Router 统一判定继续推进：去掉场景提示和硬快捷分支，让澄清、追问和查询都回到统一判定

背景：这段时间系统里仍然存在一些 router 之外的“捷径”：有的靠 scenario prompt 预先得限定场景，有的靠 shortcut 在 router 前直接改道。这种做法短期能救单点问题，但长期会让当前轮真实意图没有完整交给 router 判断，尤其容易在报告追问、车辆元数据查询和 pending clarification 恢复时产生错分流。

方案说明：本周继续沿“把旁路逻辑收回 router”这条线推进。先移除 scenario routing prompt，再把 forced resume、vehicle meta query 和 structured report follow-up 这些硬快捷分支退出执行路径，改为通过 pending clarification 上下文、最近报告上下文和工具描述共同约束 router 的判断。

功能说明：

- 删除 scenario routing prompt，减少在 router 外预先写死场景映射
- 优先透传 clarification message，让 pending further info 的恢复继续由 router 结合当前轮判断
- 移除 forced vehicle resume、vehicle meta query、structured report follow-up 三类 router intent shortcut
- 在 router prompt 中注入最近结构化报告上下文和 pending clarification 上下文，支持报告解释、重生成和换话题场景更稳定分流
- 更新 runtime/router tool descriptions，明确 consult_omni、consult_vehicle_expert 和 generate__report 的边界
- 继续收紧 scoped tool calls，减少工具越界调用和错误抢答

4 多步执行体验正式接入主链路：自然开场、报告退场与消息来源透传一起落地

背景：此前 agent 在查数或生成报告时，多数还是“先执行，后统一输出”，导致交互体验偏生硬；同时结构化消息和普通自然语言消息的来源信息透传也不够统一，前后端都容易出现“能生成，但解释不清是怎么来的”的问题。

方案说明：本周没有引入复杂 planner，而是直接在现有 worker/router/chat-service 架构外层补一层轻量多步执行编排：先生成 opening，再执行工具和 worker，最后输出正文；报告类场景在尾部再追加 closing。同时把 message sources passthrough/pipeline 统一起来，让多步执行下的来源信息继续保留下去。

功能说明：

- 后端链路支持查数/报告任务先输出 opening，再执行工具，最后返回正文；报告类任务末尾可追加 closing
- 流式与非流式都统一支持 leadingContent / trailingContent，保持前端只展示 agent 真实输出
- Router、worker、chat-service 统一透传 opening/closing，避免不同 worker 路径行为不一致
- 前端支持解析“自然语言前缀 + JSON 报告 + 自然语言后缀”，继续复用现有报告卡片渲染
- 统一 message sources passthrough 和 pipeline，保留报告、查询与咨询响应里的来源链路信息
- 为结构化消息与自然语言混排补齐兼容处理，降低展示错位和原始工具细节泄漏的风险

5 Runtime 拆分继续推进：AB 实验层和 sessions 迁出，主文件进一步收敛为组合根

背景：runtime.ts 这几周一直在做“从大而全入口退回组合根”的拆分。本周虽然用户侧可见功能不如报告链路那样集中，但结构层面的推进比较关键，因为 AB 实验、sessions 和 routing context 如果继续内联留在 Runtime 里，后续无论是改路由还是加实验都很容易相互牵连。

方案说明：本周重点把 AB 实验适配层和 sessions 相关实现继续外提。思路还是一样：先把高频变动、职责边界明确的部分迁成独立模块，再更新 runtime 快照文档，明确哪些已经不该再回到 Runtime 里直接写。

功能说明：

- 抽离 AB experiment adapter、service、metadata、types，减少 chat/router 主链路直接背负实验分组逻辑
- 新增 turn context 与相关元数据处理，统一 AB 分组、variant 和 routing mode 的传递方式
- 将 sessions repository、title service、routing context 从 Runtime 中迁出，承接会话读写、标题生成和最近 assistant routing context 读取
- 更新 runtime.md 结构快照，明确已迁出边界和下一轮应继续拆的 OpenAI/tooling 胶水
- 继续缩小 runtime.ts 对 chat、reports、sessions 的内联实现占比，保持后续演进可追踪

6 测试与交接材料继续补齐：检索服务检查包、可靠性脚本和阶段性资料一并沉淀

背景：除了主链路代码本身，这周还有一部分工作集中在“让别人能接、让后续能复盘、让问题能复现”。需要单独说明的是，本周没有把 RAG 正式接到 Agent 主问答链路，RAG 接入顺延到

了下周；本周围绕检索服务主要做的是检查包、测试服恢复说明和阶段性材料整理。如果缺少这些基础材料，后面即使开始接主链路，排查和交接成本也会很高。

方案说明:本周先把这类支撑性工作补起来,不直接改主链路接入。一方面补齐 `retrieval-latest-check` 这套独立检索服务检查包和测试服恢复说明,另一方面补齐 `assistant reliability` 的端到端 Playwright 脚本、测试用例和阶段性排查材料,让下周真正做 RAG 接入和系统验证时有现成的检查手段。

功能说明:

- 新增 `retrieval-latest-check` 独立检查包,补齐 `preview/commit/replace/retrieve/reindex/-worker` 等 KB 检索服务代码与部署说明
- 整理测试服恢复说明、Docker 启动方式和旧表兼容说明,降低 RAG 检索服务交接与恢复成本
- 新增 `assistant-reliability` 用例与 Playwright 脚本,可检查页面可用性、多轮回复、工具选择和原始 tool 泄漏
- 补充 AB、车辆报告路由召回和多用户/流式排查材料,方便继续复盘近期路由与体验问题
- 同步归档阶段性周报、实验结果和样本文档,减少后续复盘时信息分散

7 上周计划核对

核对说明: 对照 2026-04-01 周报里的下周计划,本周实际推进情况如下。

- **继续验证车辆报告链路:** 已推进。本周不只继续看车辆报告,还把驾驶员、线路主画像报告一起切到 MCP,并统一了 `prompt data sources`、最新分区和 `provider/gateway` 出口;不过稳定性验证还没有结束。
- **继续观察车辆专家 CoT 实验数据:** 部分推进。本周没有继续扩展 CoT 模式本体,主要做的是 AB experiment adapter、metadata 和 `sessions/routing context` 的拆分整理,方便后续继续观察实验数据;是否继续加大 CoT 推进力度还没有结论。
- **补齐澄清链路多轮测试:** 部分推进。本周做了 `clarification message passthrough`、`router shortcut` 移除和上下文注入,但系统化的多轮测试本身还要下周继续补。
- **继续优化 RAG 接入效果:** 本周未开展主链路接入。实际完成的是 `retrieval-latest-check` 检查包、测试服恢复说明和相关材料整理;RAG 接入顺延到下周。
- **继续处理分析页 Vue 迁移后的收尾:** 本周已完成。
- **继续推进 Runtime 拆分与文档同步:** 已推进。完成了 AB experiment adapter 抽离、`sessions` 迁出和 `runtime.md` 更新。
- **准备 MCP 服务的后备方案:** 已部分推进。本周完成了 `mock provider` 默认化、`gateway` 切换、`fixtures/samples/doc` 对齐,已经形成开发与联调层面的后备方案;但还不是生产级替代方案。

8 技术债务与限制

- **opening/closing 体验已经接入主链路,但文案稳定性仍依赖阶段 prompt:** 目前没有单独的产品化配置层,什么时候该开场、什么时候该退场,仍主要靠任务分类和 prompt 约束。

- **message sources** 虽然已经统一透传，但下游消费还没有完全统一：来源信息在 worker、报告和咨询链路里已经更一致，但前端展示、埋点和调试面板是否都能稳定消费，还需要继续看。
- **runtime.ts** 仍未拆完：sessions 和 AB 已经迁出，但 OpenAI 调用封装、query_data 执行层和 tooling 胶水仍然是后续最该继续处理的大块遗留。
- 可靠性脚本和检索检查包已经有了，但还没有进入固定测试节奏：本周更多是把工具和材料准备出来，后续还需要把它们真正纳入日常测试和交付流程。
- **RAG** 主链路接入本周没有开展，顺延到下周：本周完成的是检索服务检查包、恢复说明和兼容材料，这些准备工作不能等同于已经把 RAG 接进 Agent 问答链路；真正接入后的收益与误召回风险还要下周开始验证。

9 下周计划

1. 把所有报告链路统一成现有车辆报告的管线和表现：重点把 driver / vehicle / route 报告链路进一步收拢到现有车辆报告的实现方式上，确认底层管线、默认行为和输出表现尽量一致。
2. 正式开始 RAG 主链路接入与验证：在 retrieval-latest-check、测试服恢复说明和兼容材料的基础上，把 RAG 接到 Agent 问答链路并开始评估效果。
3. 继续观察车辆专家 CoT / AB 实验数据：结合新的实验适配层、统计材料和真实会话，判断是否继续推进 CoT 模式。
4. 继续观察 Router 统一判定后的多轮效果：重点看报告追问、车辆元数据查询和 pending clarification 恢复在复杂上下文下是否还会误分流。
5. 补齐 opening/closing 与 message sources 的端到端测试：围绕流式、非流式、结构化报告和普通咨询几个场景继续做联调。
6. 继续推进 Runtime 拆分：优先处理 OpenAI 调用封装、query_data 执行层和 tooling 胶水这些还留在 runtime.ts 里的大块遗留。
7. 把可靠性脚本和路由召回材料真正用起来：基于 Playwright、召回评估脚本和多用户样本继续做一轮系统测试，尽快发现 shortcut 移除后的新问题。